

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №2

з дисципліни
«Об'єктно-орієнтоване програмування»
на тему
“Розробка графічного редактора об'єктів”

Виконав:

студент групи ІМ-о51
Тараєв Олексій Богданович
Номер в списку - 14

Перевірів:

Рекечинський Д.О.

Київ 2026

Завдання:

- 1 Створити у середовищі розробки проект з ім'ям Lab2.
- 2 Написати вихідний текст програми об'єктно-орієнтованого векторного графічного редактора відповідно до варіанта завдання.
- 3 Отримати виконуваний файл програми та провести налагодження логіки введення геометричних фігур.
- 4 Перевірити роботу механізмів малювання, поліморфного відображення об'єктів та обробки подій маніпулятора «миша».
- 5 Оформити звіт із діаграмою класів та ілюстраціями результатів роботи програми.

Завдання згідно варіанту (Варіант 14)

Розрахунок варіанта (Варіант 14)

Номер студента у списку: $\mathcal{K} = 14$.

Масив покажчиків `Shape`: статичний масив на $N = 14 + 100 = 114$ об'єктів

«Гумовий» слід: суцільна лінія синього кольору ($14 \bmod 4 = 2$).

Прямокутник: введення по двох протилежних кутах ($14 \bmod 2 = 0$), чорний контур без заповнення ($14 \bmod 5 = 4$).

Еліпс: введення від центру до одного з кутів ($14 \bmod 2 = 0$), чорний контур зі світло-зеленим заповненням ($14 \bmod 5 = 4$, $14 \bmod 6 = 2$).

Позначення поточного режиму: позначка в меню через `OnInitMenuPopup` ($14 \bmod 2 = 0$).

Зміст завдань:

Розробити віконну програму векторного графічного редактора з головним меню, де меню **"Об'єкти"** розміщене праворуч від меню **"Файл"** та ліворуч від меню **"Довідка"**. Меню повинно містити підпункти українською мовою: «Крапка», «Лінія», «Прямокутник», «Еліпс».

Забезпечити позначення поточного вибраного типу фігури позначкою (галочкою/радіокнопкою) безпосередньо у випадаючому меню.

Реалізувати ієрархію геометричних фігур на основі базового абстрактного класу `Shape` та класів-нащадків `PointShape`, `LineShape`, `RectShape`, `EllipseShape` з поліморфним методом `show()`.

Реалізувати ієрархію обробників введення на основі базового класу `Editor` та `ShapeEditor` із класами-нащадками `PointEditor`, `LineEditor`, `RectEditor`, `EllipseEditor`, що обробляють натискання, рух та відпускання лівої кнопки миші.

Під час малювання ліній, прямокутників та еліпсів забезпечити відображення «гумового» сліду синього кольору, який динамічно слідує за вказівником миші до моменту фіксації фігури.

Зберігання створених фігур організувати у статичному масиві фіксованого розміру $N = 114$. Перемалювання полотна має відбуватися циклічним викликом методу `show()` для кожного збереженого екземпляра без перевірки його конкретного типу (використання поліморфізму).

КОД

[shape.py](#)

```
import tkinter as tk
```

```
from abc import ABC, abstractmethod
```

```
__all__ = ["EllipseShape", "LineShape", "PointShape", "RectShape",  
"Shape"]
```

```
class Shape(ABC):
```

```
    def __init__(self) -> None:
```

```
        self.xs1: int = 0
```

```

        self.ys1: int = 0

        self.xs2: int = 0

        self.ys2: int = 0

    def set(self, x1: int, y1: int, x2: int, y2: int) -> None:

        self.xs1 = x1

        self.ys1 = y1

        self.xs2 = x2

        self.ys2 = y2

    @abstractmethod

    def show(self, canvas: tk.Canvas) -> None:

        pass


class PointShape(Shape):

    def show(self, canvas: tk.Canvas) -> None:

        canvas.create_oval(

            self.xs1 - 1,

            self.ys1 - 1,

            self.xs1 + 1,

            self.ys1 + 1,

            fill="black",

            outline="black",

            tags="shape",

        )


class LineShape(Shape):

```

```
def show(self, canvas: tk.Canvas) -> None:

    canvas.create_line(

        self.xs1,

        self.ys1,

        self.xs2,

        self.ys2,

        fill="black",

        width=1,

        tags="shape",

    )
```

```
class RectShape(Shape):
```

```
    def show(self, canvas: tk.Canvas) -> None:

        x_min = min(self.xs1, self.xs2)

        y_min = min(self.ys1, self.ys2)

        x_max = max(self.xs1, self.xs2)

        y_max = max(self.ys1, self.ys2)

        canvas.create_rectangle(

            x_min,

            y_min,

            x_max,

            y_max,

            fill="",

            outline="black",

            width=1,

            tags="shape",

        )
```

```
class EllipseShape(Shape):

    def show(self, canvas: tk.Canvas) -> None:

        x_min = min(self.xs1, self.xs2)

        y_min = min(self.ys1, self.ys2)

        x_max = max(self.xs1, self.xs2)

        y_max = max(self.ys1, self.ys2)

        canvas.create_oval(

            x_min,

            y_min,

            x_max,

            y_max,

            fill="#90EE90",

            outline="black",

            width=1,

            tags="shape",

        )
```

editor.py

```
import logging
```

```
import tkinter as tk
```

```
from abc import ABC, abstractmethod
```

```
from typing import Any
```

```
from shape import EllipseShape, LineShape, PointShape, RectShape
```

```
__all__ = [
    "Editor",
    "EllipseEditor",
    "LineEditor",
    "PointEditor",
    "RectEditor",
    "ShapeEditor",
]

logger = logging.getLogger("Lab2.Editor")

class Editor(ABC):
    @abstractmethod
    def on_lb_down(self, event: tk.Event) -> None:
        pass

    @abstractmethod
    def on_lb_up(self, event: tk.Event) -> None:
        pass

    @abstractmethod
    def on_mouse_move(self, event: tk.Event) -> None:
        pass

    @abstractmethod
    def on_paint(self, canvas: tk.Canvas) -> None:
        pass
```

```
class ShapeEditor(Editor):

    def __init__(self, owner: Any) -> None:

        self._owner = owner

        self._x_start: int = 0

        self._y_start: int = 0

        self._x_end: int = 0

        self._y_end: int = 0

        self._is_drawing: bool = False

    def on_paint(self, canvas: tk.Canvas) -> None:

        pass

    def _clear_rubber_band(self) -> None:

        self._owner.canvas.delete("rubber_band")

class PointEditor(ShapeEditor):

    def on_lb_down(self, event: tk.Event) -> None:

        logger.debug("PointEditor mouse down at (%d, %d).", event.x,
event.y)

        shape = PointShape()

        shape.set(event.x, event.y, event.x, event.y)

        self._owner.add_shape(shape)

    def on_lb_up(self, event: tk.Event) -> None:

        pass

    def on_mouse_move(self, event: tk.Event) -> None:
```

```
pass

class LineEditor(ShapeEditor):

    def on_lb_down(self, event: tk.Event) -> None:

        logger.debug("LineEditor start drag at (%d, %d).", event.x,
event.y)

        self._is_drawing = True

        self._x_start = event.x

        self._y_start = event.y

        self._x_end = event.x

        self._y_end = event.y


    def on_mouse_move(self, event: tk.Event) -> None:

        if not self._is_drawing:

            return

        self._x_end = event.x

        self._y_end = event.y

        self._clear_rubber_band()

        self._owner.canvas.create_line(

            self._x_start,

            self._y_start,

            self._x_end,

            self._y_end,

            fill="blue",

            width=1,

            tags="rubber_band",

        )
```

```

def on_lb_up(self, event: tk.Event) -> None:

    if not self._is_drawing:

        return

    logger.debug("LineEditor end drag at (%d, %d).", event.x,
event.y)

    self._is_drawing = False

    self._clear_rubber_band()

    shape = LineShape()

    shape.set(self._x_start, self._y_start, event.x, event.y)

    self._owner.add_shape(shape)


class RectEditor(ShapeEditor):

    def on_lb_down(self, event: tk.Event) -> None:

        logger.debug("RectEditor start drag at (%d, %d).", event.x,
event.y)

        self._is_drawing = True

        self._x_start = event.x

        self._y_start = event.y

        self._x_end = event.x

        self._y_end = event.y


    def on_mouse_move(self, event: tk.Event) -> None:

        if not self._is_drawing:

            return

        self._x_end = event.x

        self._y_end = event.y

        self._clear_rubber_band()

        x_min = min(self._x_start, self._x_end)

        y_min = min(self._y_start, self._y_end)

```

```

        x_max = max(self._x_start, self._x_end)
        y_max = max(self._y_start, self._y_end)

        self._owner.canvas.create_rectangle(

            x_min,

            y_min,

            x_max,

            y_max,

            fill="",

            outline="blue",

            width=1,

            tags="rubber_band",

        )

    def on_lb_up(self, event: tk.Event) -> None:

        if not self._is_drawing:

            return

        logger.debug("RectEditor end drag at (%d, %d).", event.x,
event.y)

        self._is_drawing = False

        self._clear_rubber_band()

        shape = RectShape()

        shape.set(self._x_start, self._y_start, event.x, event.y)

        self._owner.add_shape(shape)

class EllipseEditor(ShapeEditor):

    def on_lb_down(self, event: tk.Event) -> None:

        logger.debug("EllipseEditor center placed at (%d, %d).",
event.x, event.y)

        self._is_drawing = True

```

```

        self._x_start = event.x

        self._y_start = event.y

        self._x_end = event.x

        self._y_end = event.y


def on_mouse_move(self, event: tk.Event) -> None:

    if not self._is_drawing:

        return

    self._x_end = event.x

    self._y_end = event.y

    self._clear_rubber_band()

    radius_x = abs(self._x_end - self._x_start)

    radius_y = abs(self._y_end - self._y_start)

    self._owner.canvas.create_oval(

        self._x_start - radius_x,

        self._y_start - radius_y,

        self._x_start + radius_x,

        self._y_start + radius_y,

        fill="",

        outline="blue",

        width=1,

        tags="rubber_band",

    )


def on_lb_up(self, event: tk.Event) -> None:

    if not self._is_drawing:

        return

    logger.debug("EllipseEditor corner released at (%d, %d).",

event.x, event.y)

```

```
self._is_drawing = False

self._clear_rubber_band()

radius_x = abs(event.x - self._x_start)
radius_y = abs(event.y - self._y_start)

shape = EllipseShape()

shape.set(

    self._x_start - radius_x,

    self._y_start - radius_y,

    self._x_start + radius_x,

    self._y_start + radius_y,

)

self._owner.add_shape(shape)
```

shape_objects_editor.py

```
import logging

import tkinter as tk

from editor import (

    EllipseEditor,

    LineEditor,

    PointEditor,

    RectEditor,

    ShapeEditor,

)

from shape import Shape

__all__ = ["ShapeObjectsEditor"]
```

```
logger = logging.getLogger("Lab2.ShapeObjectsEditor")
```

```
class ShapeObjectsEditor:
```

```
    ARRAY_CAPACITY = 114
```

```
    def __init__(self, canvas: tk.Canvas) -> None:
```

```
        self.canvas = canvas
```

```
        self._pcshape: list[Shape | None] = [None] *  
self.ARRAY_CAPACITY
```

```
        self._count: int = 0
```

```
        self._editors: dict[str, ShapeEditor] = {
```

```
            "point": PointEditor(self),
```

```
            "line": LineEditor(self),
```

```
            "rect": RectEditor(self),
```

```
            "ellipse": EllipseEditor(self),
```

```
        }
```

```
        self._current_type_name: str = "point"
```

```
        self._current_editor: ShapeEditor =  
self._editors[self._current_type_name]
```

```
    @property
```

```
    def current_type_name(self) -> str:
```

```
        return self._current_type_name
```

```
    def start_editor(self, shape_type: str) -> None:
```

```
        if shape_type in self._editors:
```

```

        logger.info("Switching mode to %s.", shape_type)

        self._current_editor = self._editors[shape_type]

        self._current_type_name = shape_type

    else:

        logger.error("Unknown editor type requested: '%s'.",
shape_type)

def on_lb_down(self, event: tk.Event) -> None:

    self._current_editor.on_lb_down(event)

def on_lb_up(self, event: tk.Event) -> None:

    self._current_editor.on_lb_up(event)

def on_mouse_move(self, event: tk.Event) -> None:

    self._current_editor.on_mouse_move(event)

def on_paint(self) -> None:

    self.canvas.delete("shape")

    for i in range(self._count):

        shape = self._pcshape[i]

        if shape is not None:

            shape.show(self.canvas)

def add_shape(self, shape: Shape) -> None:

    if self._count < self.ARRAY_CAPACITY:

        self._pcshape[self._count] = shape

        self._count += 1

        logger.info(

            "Shape added. Current array size: %d/%d.",

```

```

        self._count,

        self.ARRAY_CAPACITY,

    )

    self.on_paint()

    else:

        logger.warning("Shape array capacity reached (%d).",
self.ARRAY_CAPACITY)

    def on_init_menu_popup(self, menu_var: tk.StringVar) -> None:

        menu_var.set(self._current_type_name)

        logger.debug("Menu state synced: current mode '%s'.",
self._current_type_name)

```

main.py

```

import logging

import tkinter as tk

from tkinter import messagebox

from shape_objects_editor import ShapeObjectsEditor

logging.basicConfig(

    level=logging.DEBUG,

    format="%(asctime)s [%(levelname)s] (%(name)s): %(message)s",

    datefmt="%H:%M:%S",

)

logger = logging.getLogger("Lab2.Main")

```

```
class MainWindow(tk.Tk):

    def __init__(self) -> None:

        super().__init__()

        logger.info("Initializing MainWindow.")

        self.title("OOP_lab2")

        self.geometry("700x500")

        self._menu_shape_var = tk.StringVar(value="point")

        self._init_canvas()

        self._editor = ShapeObjectsEditor(self._canvas)

        self._init_menu()

        self._bind_events()

    def _init_canvas(self) -> None:

        self._canvas = tk.Canvas(self, bg="#FFFFFF",
cursor="crosshair")

        self._canvas.pack(fill=tk.BOTH, expand=True)

    def _init_menu(self) -> None:

        logger.debug("Configuring menu bar.")

        menubar = tk.Menu(self)

        file_menu = tk.Menu(menubar, tearoff=0)

        file_menu.add_command(label="Вихід", command=self.destroy)

        menubar.add_cascade(label="Файл", menu=file_menu)

        objects_menu = tk.Menu(

            menubar,
```

```
        tearoff=0,

        postcommand=lambda:
self._editor.on_init_menu_popup(self._menu_shape_var),

    )

    objects_menu.add_radiobutton(

        label="Крпка",

        variable=self._menu_shape_var,

        value="point",

        command=lambda: self._editor.start_editor("point"),

    )

    objects_menu.add_radiobutton(

        label="Линия",

        variable=self._menu_shape_var,

        value="line",

        command=lambda: self._editor.start_editor("line"),

    )

    objects_menu.add_radiobutton(

        label="Прямокутник",

        variable=self._menu_shape_var,

        value="rect",

        command=lambda: self._editor.start_editor("rect"),

    )

    objects_menu.add_radiobutton(

        label="Елпс",

        variable=self._menu_shape_var,

        value="ellipse",

        command=lambda: self._editor.start_editor("ellipse"),

    )

    menubar.add_cascade(label="Об'єкти", menu=objects_menu)
```

```

        help_menu = tk.Menu(menubar, tearoff=0)

        help_menu.add_command(label="Про програму",
command=self._on_about)

        menubar.add_cascade(label="Довідка", menu=help_menu)

        self.config(menu=menubar)

def _bind_events(self) -> None:

    self._canvas.bind("<Button-1>", self._editor.on_lb_down)

    self._canvas.bind("<B1-Motion>", self._editor.on_mouse_move)

    self._canvas.bind("<ButtonRelease-1>", self._editor.on_lb_up)

    self._canvas.bind("<Configure>", lambda _event:
self._editor.on_paint())

def _on_about(self) -> None:

    logger.info("Displaying About dialog.")

    messagebox.showinfo(

        "Про програму",

        "Лабораторна робота №2\nГрафічний редактор
об'єктів\nВаріант 14 (N=114)",

    )

def main() -> None:

    logger.info("Starting application loop.")

    app = MainWindow()

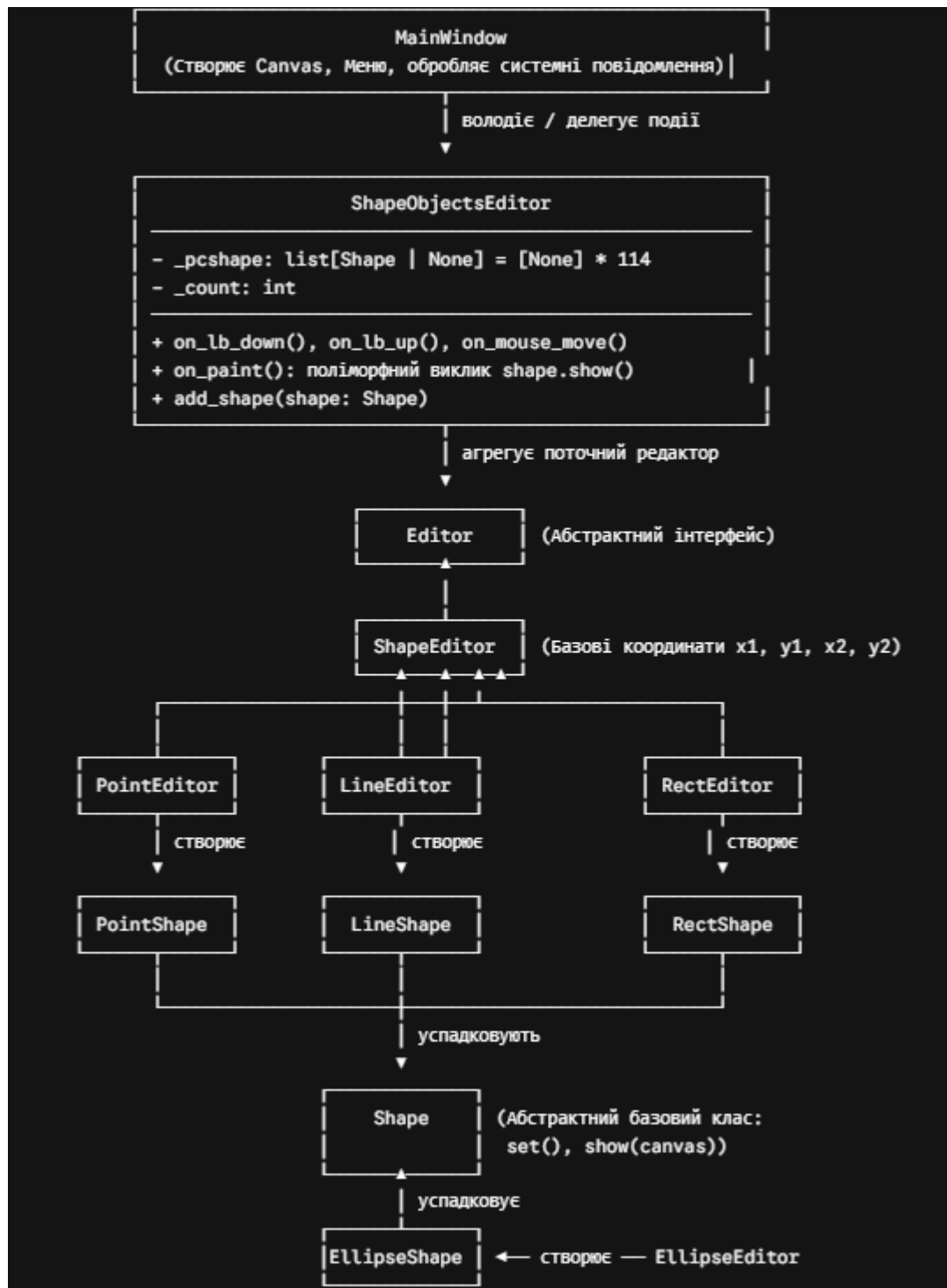
    app.mainloop()

    logger.info("Application loop ended.")

```

```
if __name__ == "__main__":  
    main()
```

Діаграма класів та залежностей



Клас **MainWindow** ініціалізує графічну оболонку та передає повідомлення від миші до єдиного контролера **ShapeObjectsEditor**.

ShapeObjectsEditor зберігає посилання на поточний активний інструмент введення (**ShapeEditor**) та масив фіксованої довжини

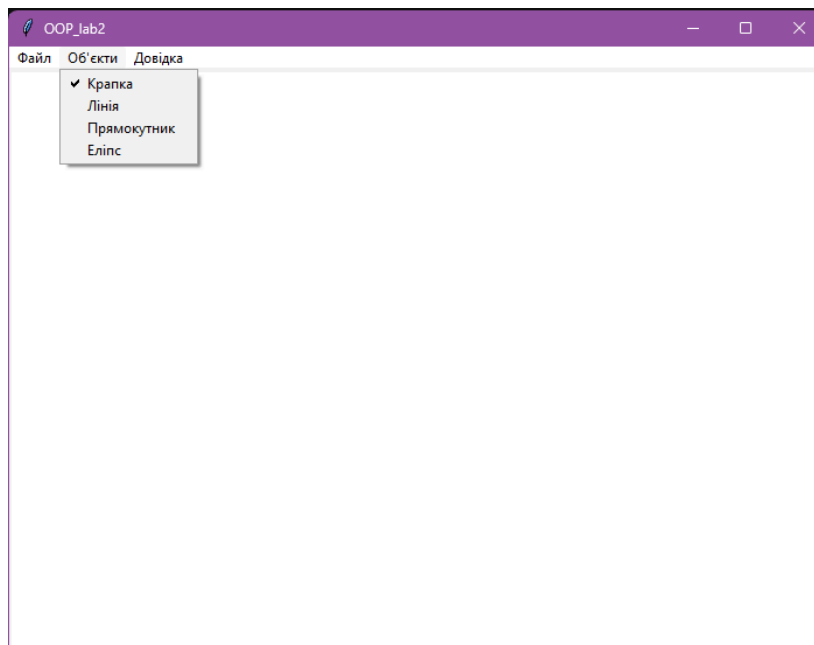
$N = 114$ для об'єктів типу **Shape**.

Конкретні редактори (**PointEditor**, **LineEditor**, **RectEditor**, **EllipseEditor**) обробляють координати подій, малюють тимчасовий синій слід та після відпускання кнопки миші інстанціюють відповідну фігуру.

Додана фігура потрапляє у статичний масив **_pcshape**, після чого метод **on_paint()** здійснює перемалювання полотна через єдиний поліморфний інтерфейс **shape.show(canvas)**

скріншоти

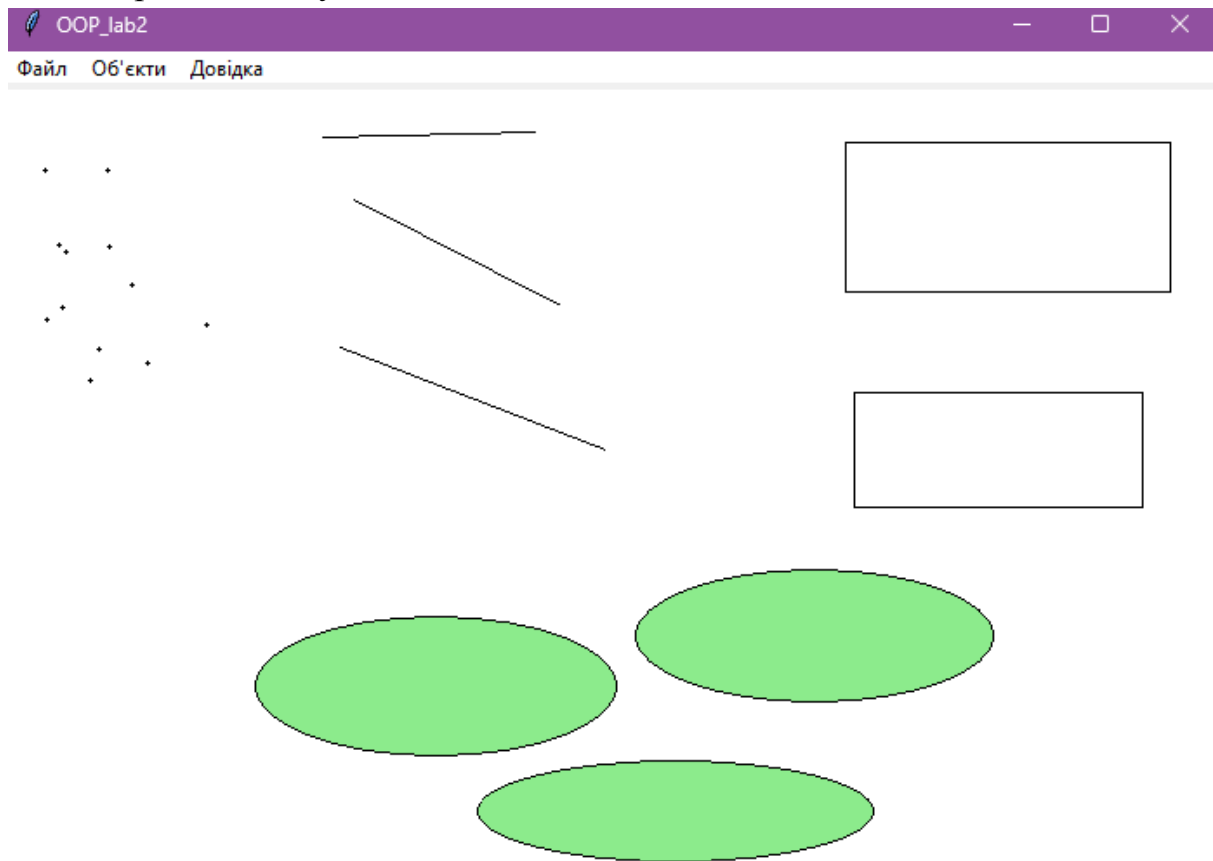
головне меню:



процес введення об'єкта з відображенням синього «гумового» сліду



демонстрація виводу всіх об'єктів



Висновки

Під час виконання лабораторної роботи №2 було спроектовано та реалізовано об'єктно-орієнтований векторний графічний редактор мовою Python.

У ході роботи було практично застосовано фундаментальні принципи ООП:

Абстракція та інкапсуляція: створено базові абстрактні класи **Shape** та **Editor**, які відокремлюють інтерфейс взаємодії від внутрішнього стану та деталей відмалювання віджетів. Стан координат, режим протягування та структури збереження повністю приховані всередині відповідних класів.

Успадкування: побудовано дві паралельні ієрархії класів — графічних примітивів (**Shape**) та відповідних інструментів введення (**ShapeEditor**), що забезпечує чітке розмежування відповідальностей за принципом Single Responsibility.

Поліморфізм: реалізовано механізм динамічної диспетчеризації методів: метод перемалювання **on_paint()** взаємодіє з масивом покажчиків базового типу **Shape** через абстрактний метод **show()**, що дозволяє рендерити примітиви з різними правилами геометрії та заливки (порожній прямокутник, залитий еліпс) без розгалужень типу **switch/case** чи перевірок типів